

别再让一个 AI 包揽全场 PaperRoute 多模型分工做游戏方法论全解

一条推文撕开的底牌：四天、15.6 亿 Token、98% 是缓存读取

案例主角：3D 送报游戏 **PaperRoute** (作者 Emm Tee / @BuiltBySketch, 浏览器直接玩 paperroute.lol)

资料来源：AYi(@AYi_Alnotes) 拆解推文 + 作者公开开发日志 paperroute.lol/devlog (69 个 checkpoint 逐节点截图)

学习日期：2026-10-03 · 拾遗任务 7d3909c7 · 配套实验见 [exercise\目录](#)

1.56B

总 Token (98% 缓存)

39h

追踪工时 (25.2 人 + 13.8 纯agent)

90

commits / 69 checkpoint

76.9万

行变更 / 4458 文件

3

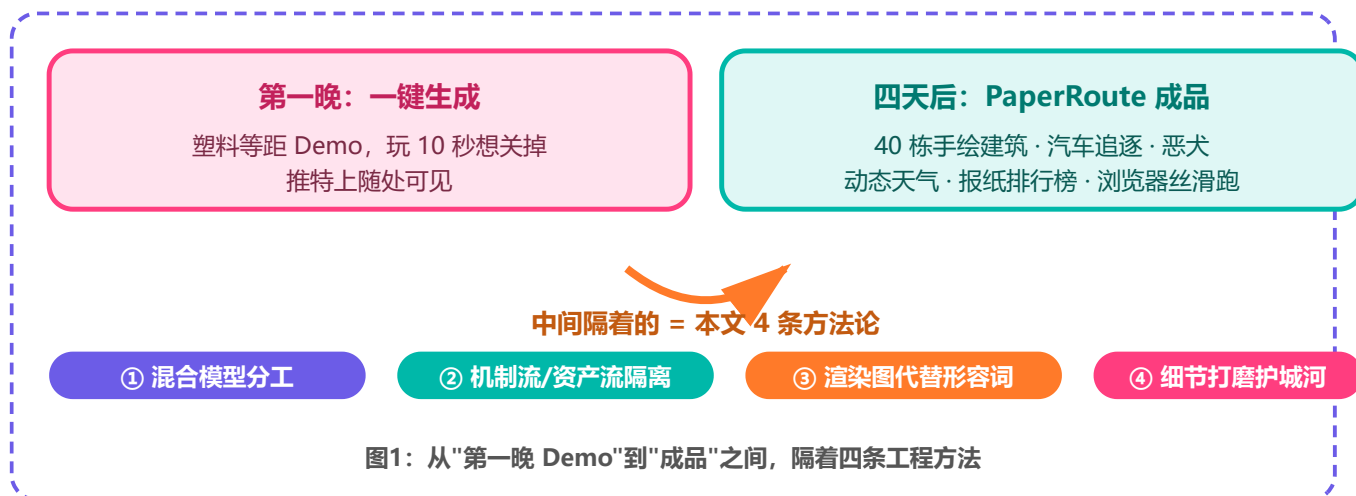
个复现实验全过

一句话总结

大模型只能替你完成第一晚的 Demo 狂欢；把零件磨成“愿意玩第二次”的作品，靠的是拆任务、建测试、隔离实验的工程克制——而这套克制，完全可以学。

Q1这条推文到底在拆什么底牌？

AYi 转发并拆解了独立开发者 **Emm Tee (@BuiltBySketch)** 的帖子：他用 GPT-5.6 Astra 在浏览器里做了一个 3D 送报游戏 **PaperRoute**——骑自行车沿街投报，躲汽车、恶犬和坑洞，40 栋手绘建筑、动态天气、复古报纸结算 UI。**关键不是"AI 做了个游戏"，而是他公开了完整开发日志：69 个带截图的开发节点。**真相是：第一晚的"一键生成"只吐出一个玩十秒就想关的塑料等距 Demo，从 Demo 到成品靠的是四条冷血的工业级分工。



Q2"15.6 亿 Token，98% 是缓存读取"说明什么？

开发日志的统计面板给了一手数字：**1.56B 总 Token = 26.2M 输入 + 6.0M 输出 + 1.53B 缓存读取**。真正"生成"的只有 6M，**其余 98% 是 AI 反复读旧上下文**。这说明 AI 开发的成本大头根本不是"写字"，而是**把项目背景反复塞回模型脑子里**——这正是后面"隔离对话、少塞无关背景"的方法论根源：你少塞一点无关上下文，就是在省钱省时间。



Q3为什么"一个 AI 包揽全场"必死在穿模和屎山里？

推文里的灵魂拷问：为什么你用大模型做游戏跑两步就死在穿模和屎山代码里？因为你逼着同一个 AI 既当程序员又当插画师。**单一对话塞进所有角色，上下文过载、角色混乱，一错到底**——Astra 写玩法物理是天才，但让它捏带脸的主角，"烧光额度做出来的也是个僵尸木偶"。这和我们知识库 [[多智能体协作]] 的结论完全同源：拆开不是为了酷，是因为每个上下文只装一个角色才专注、才能互查。

 知识库关联：[[多智能体协作]]（拆=指令专注+工具专管）、[[能力地板选型]]（按任务的能力地板选最便宜的生产者，不按模型天花板付钱）、[[上下文预算与战略压缩]]（上下文是稀缺资源）。

Q4混合模型流水线具体怎么路由？（方法论①）

作者拉了一条极其务实的混合流水线，每类任务只交给"干这个最便宜且最好"的工具：

任务类型	生产者	为什么是它
玩法逻辑 / 物理碰撞 / 绑骨	Astra（代码旗舰）	它的天才区；消除自行车穿模、绑 23 根骨骼
角色概念图	ChatGPT	出图便宜快速，方向对了再往下走
带脸的 3D 角色网格	Meshy（专用生成，8 美元/个）	专用工具一条龙，比烧旗舰额度便宜几个数量级
批量建筑资产	Astra 写脚本 → Headless Blender 批产	程序化生成 40 栋，模型只写脚本不画模型
动作审查	Astra + 自动截图	看渲染图，不听形容词（Q7）

核心原则：路由的单位是"任务类型"，不是"项目"。同一个游戏里，代码给 Astra、网格给 Meshy、概念图给 ChatGPT——最后还要"拉回 Astra 绑骨骼"，因为集成永远回到主车道。

 实验复现（E2·exercise\task_router.py）：把 devlog 里 12 条真实任务喂给规则路由器，**与作者实际选择一致率 12/12**；全旗舰成本系数 12.00 → 按车道路由后 6.90（1.7x，网格类若自研则差距拉到数量级）。

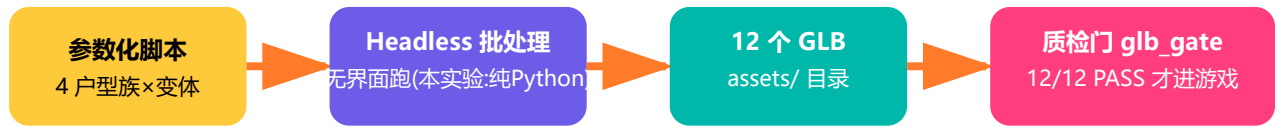
Q5机制流与资产流为什么要物理隔离？（方法论②）

作者最痛的领悟："房子难看时，千万不要在同一段对话里跟模型吵抛物线物理！"他把工作劈成互不污染的两半：**机制流（手感/物理）**：一个 Prompt → 浏览器看一眼 → 提交一次 Commit；**资产流（美术）**：AI 写 Python 脚本 → Headless 后台批量生成 GLB。两套逻辑物理隔离，"手感出了问题绝不去改屋顶"。这就是 [[GitWorktree并行隔离]] 的思想用在"对话上下文"上：隔离的单位不是目录，是**话题上下文本身**。



Q6Headless 资产管线到底怎么工作?

devlog checkpoint 16 的原话标题: "Replace playable house bodies with authored Blender asset families", 并且"rider .blend + asset pipeline scripts"作为资产入库到 [ArtSource/](#)。模式是: **让 AI 写 Python 脚本, 无人值守跑 Blender, 批量吐 GLB 给游戏加载**。人(和主对话)只面对"脚本 + 产物", 不面对 40 栋房子逐个手摆。本机没装 Blender 也能复刻这个模式——GLB 就是个二进制容器(JSON 描述 + 几何数据), 纯 Python 手写一个"房屋工厂"完全可行。



打回 = 资产流自己重跑脚本; 机制流永远看不到失败品(隔离的意义)

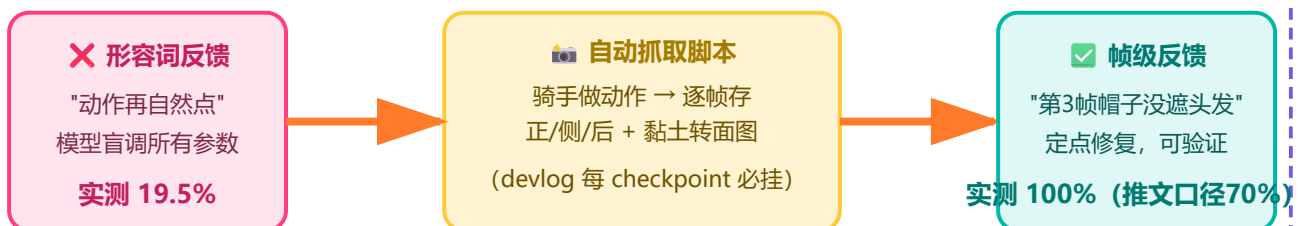
图4: Headless 资产管线微缩版——与 devlog 的 Blender 批产同构

🔗 实验复现 (E1 · exercise\house_factory.py + glb_gate.py) : `python house_factory.py` 纯 Python 手写 GLB 二进制 (magic glTF / JSON chunk / BIN chunk), 4 族×3 栋 = 12 个文件全生成 (1208~1216 bytes); `python glb_gate.py` 质检门校验魔数、版本、块长度自洽、JSON 可解析、POSITION accessor、三角索引——**12/12 PASS**。

📖 知识库关联: [[程序化建模]] (用代码生成模型)、[[glTF与GLB资产管线]] (GLB=真空压缩袋, 几何与装配分离)。

Q7"用渲染图代替形容词"为什么有效? (方法论③)

对着对话框说"把动作改得更自然点", 模型听不懂这种玄学。作者的做法: 搭自动抓取脚本, 让骑手在场景里做动作, **每次自动保存正、侧、后 + 黏土材质转面图** (devlog 里每个 checkpoint 都挂着这些 Development renders)。然后指着具体帧提意见:"帽子第 3 帧没遮住头发"——改动成功率直接飙到 70%。本质: 把**主观形容词**翻译成**带定位坐标的运行时证据**, 反馈才可执行。这正是知识库 [[证据优先质检ProofOverClaims]] 的"视觉质检闭环": 验收标准是"亲眼看到产物", 且证据要**可定位**。



E3 实验 (frame_review_loop.py, 200 轮): 形容词 19.5% vs 帧索引 100%, 5.1 倍

图5: 形容词 vs 帧级证据——反馈里每多一个可定位坐标, 成功率涨一截

Q8从 0 到可玩，工程顺序是什么？

devlog 的时间线（7 月 iOS 期试水失败 → 9 月 Browser 期 4 天冲刺）第一个 commit 就是：P0 baseline: PRD, engineering standards, architecture contract——先写产品需求文档、工程标准、架构契约，此时连可运行的应用都没有。然后才动工：

阶段	devlog 证据	对应知识库
P0 基线	PRD + 工程标准 + 架构契约（无可运行应用）	[[规格书先行SpecDriven]] / [[项目宪法Constitution]]
P0 感觉原型	ride + throw + dog + 标志性镜头，灰盒 Easy Street	[[纯函数游戏引擎与种子复放]]
P1 垂直切片	周一整天流程 + 障碍赛道 + 报纸结算，风格化视觉	[[质检Gate与自我纠错循环]]
P2 艺术层	插画风 / 授权 3D 骑手 / 街头镜头（docs: art-direction study of Abeto's Messenger）	[[角色一致性锚定]]
交接纪律	"P2-art handoff notes, next-session pick-up order"——每次收工写下下次从哪捡起	[[Checkpoint存档与持久执行]]

值得抄的一条：“next-session pick-up order”——AI 项目的会话是易失的，每次收工给下一个会话留“从哪继续”的交接单，等于给上下文做了持久化。

Q969 个 checkpoint + 截图 + SHA-256 是什么工作习惯？

devlog 把 69 个 checkpoint 全部归档：每条含时间戳（精确到分钟）、commit 短哈希、一句话说明、桌面/竖屏双截图、正侧后 Development renders、甚至文件 SHA-256。另有 83 张开发渲染图。7 Sep 的记录里还能看到"2 separate branch experiments, since reviewed and folded into the main game"——分支实验先隔离验证，再合并回主线。这不是仪式感：每个 checkpoint 都是给“下一次 AI 会话”的证据锚点——AI 不需要重读全部历史，只需读最近几个 checkpoint 就能接上工作。这就是 [[证据状态机]] 和 [[证据优先质检 ProofOverClaims]] 的项目级实践。

低成本复制法：不需要做网站，只要约定每个 commit 三件套——一句话说明 + 一张运行截图 + 可复核的哈希/版本号。AI 的下个会话、未来的你、协作者，全都受益。

Q1025.2h 人类 + 13.8h 纯 agent，怎么组织的？

统计面板还有一组惊人数字：**追踪覆盖 39h = 25.2h 人在场 + 13.8h 无人在键盘前**；agent 运行时（并行叠加）30h15m。在开始计时后的实测窗口里，**5.1h 人工对 10.4h 纯 agent——三分之二的时间没人在场**。这正是“机制流一个 Prompt 一提交”的副产品：任务被拆得足够小、验收被定义得足够客观（截图 + 测试门）之后，agent 才敢被放着独自跑。人的角色从“操作员”变成“验收员 + 方向盘”。

39h 追踪覆盖的构成

人在场 25.2h (含 6 Sep 前默认计入的 ~14h)

纯 agent 13.8h

开启在场统计后：实测窗口 5.1h 人工 : 10.4h 纯 agent —— 任务拆小 + 验收客观，agent 才能 solo

对应推文第 4 条：**80% 工时花在细节打磨——人类时间该花在品味上，不是打字上**

图6：人的时间用于“方向盘 + 验收”，agent 时间用于“里程”

Q11真正的壁垒为什么是细节打磨？（方法论④）

推文最后一条最扎心：整套 39 工时里，**80% 花在碎玻璃特写运镜、雨天轮胎打滑水印、终点自由练车公园、复古报纸结算 UI 这些“极其繁琐的手工活”上**。大模型降低的是“把零件拼出来”的门槛；能不能拼出“愿意玩第二次”的作品，看的是创作者带进来的品味与工程克制。用我们知识库的话说：AI 把“生成”变成了地板块（[[源码即 Prompt]] 反面——生成物廉价化），**稀缺的变成了“判断哪个生成物是对的”**。

对个人的操作含义：学 AI 开发，别练“怎么写出一条神 Prompt”，要练三件事——**拆任务的刀法（Q4/Q5）、建验收的眼力（Q7/Q9）、扛打磨的耐心（本条）**。

Q12我能带走什么？能力清单 + 上手路线

🎓 能力清单（这套方法论给你的）

- 按"任务类型"给 AI 排生产流水线，而不是找"一个全能 AI"
- 机制/资产双泳道隔离对话，杜绝上下文互相污染
- 无人值守批产资产：脚本 → 批量产物 → 质检门
- 用可定位证据（帧号/截图/哈希）代替形容词反馈
- P0 先立 PRD/工程标准/架构契约，再开工
- checkpoint 三件套 + 交接单，让下个 AI 会话秒接上

✂️ 你能用它干什么

- 做自己的"能玩"的小游戏/3D 小品（Three.js + 批产 GLB）
- 任何"AI 生成 + 人工验收"的生产线：图集、文档、课件
- 管好自己的多会话 AI 项目：隔离 + 交接单 + 证据归档
- 给团队立"AI 施工规范"：P0 基线 + 质检门 + checkpoint



每一步都遵守：一个 Prompt 一看一提交 · 每步留下可复核证据

图7：可直接照抄的四步上手路线

🔧 动手实验区（exercise\ 目录，全部真跑通）

实验 E1 • Headless 资产管线微缩版（主实验）

复刻 devlog "AI 写脚本 Headless 批产建筑 GLB + 质检门"。本机无 Blender，用纯 Python 手写 GLB 二进制，模式完全同构。

步骤 1： 打开终端，`cd F:\reminder\paperroute\exercise`。目的：进入实验目录。

步骤 2： `python house_factory.py`。目的：跑资产流批产。实现：参数化 4 户型族×3 变体，手写 GLB (glTF magic + JSON chunk + BIN chunk)。

✅ **预期结果：** 打印 12 个 `house_*.glb`（各约 1.2KB）到 `assets\`，如 `house_cottage_1.glb` 1216 bytes。

步骤 3： `python glb_gate.py`。目的：跑质检门（机制流只消费通过门的产品）。实现：校验魔数 0x46546C67、版本 2、块长度自洽、JSON 可解析、POSITION accessor、三角索引非空。

✅ **预期结果：** 12 行 `PASS` + 末行 `[glb_gate] 12/12 通过质检门`，退出码 0。

步骤 4（改造玩）： 把 `house_factory.py` 里 `FAMILIES` 的 `roof_rise` 调大、`VARIANTS = 5`，重跑两脚本。目的：体会"改参数=改整批资产"的程序化威力。

✅ **预期结果：** 出现 20 个 GLB 且全 PASS；别墅屋顶明显变尖（用任意 glTF 查看器或读 accessor max 值验证 y 变大）。

实验 E2 • 混合模型任务路由器

`python task_router.py`。目的：验证"按任务类型路由 ≠ 单一旗舰包揽"。实现：关键词规则把 12 条 devlog 真实任务分到 code/concept/mesh3d/asset/review 五条车道，并对比两种成本。

✅ **预期结果：** 输出"路由与作者实际选择一致率: 12/12"、成本系数 12.00 → 6.90 (1.7x)。

改造：往 TASKS 里加你自己的任务（如"生成 boss 战音乐"），观察它落进哪条车道——你会立刻发现规则路由的局限，这正是真实系统要用小模型做语义路由的原因。

实验 E3 • 渲染图代替形容词审查环

`python frame_review_loop.py`。目的：复现"帧级反馈 vs 形容词反馈"的成功率差。实现：24 帧骑行动画在第 3 帧注入帽子偏移缺陷，对比两种反馈模式的 200 轮修复成功率。

✅ **预期结果：**形容词反馈 19.5% vs 帧索引反馈 100% (5.1 倍)，与推文"改动成功率飙到 70%"方向一致。

📌 一页记忆卡

- **成本真相：**1.56B Token 里 98% 是缓存读取 → 上下文是最大开销，隔离即省钱
- **路由：**代码→Astra，概念图→ChatGPT，网格→Meshy(\$8)，批产→Headless 脚本；集成回主车道
- **隔离：**"房子难看时，别在同一段对话里吵抛物线物理"
- **反馈：**给帧号不给形容词；正/侧/后转面图是审查语言
- **基线：**第一个 commit 是 PRD+工程标准+架构契约，代码为零
- **护城河：**39 工时 80% 在细节打磨——品味不可代工

资料：AYi 推文 @AYi_Alnotes/status/2099090127452524615 · Emm Tee @BuiltBySketch (游戏 paperroute.lol / 日志 paperroute.lol/devlog) · 游戏本体未开源，本指南为方法论拆解 + 方法层复现。